

Agent Orchestration

Checks, handoffs and shared skills

Sergei Telegin

github.com/TeleginS/agent-orchestration

157 tests passed. QA blocked the PR

TrafficRulesApp: my iOS app for the Spanish driving theory exam

32 new tests

Review approved
the change

QA questioned what
the tests proved

QA — Pass 1

Verdict:  BLOCKED on three in-scope findings.

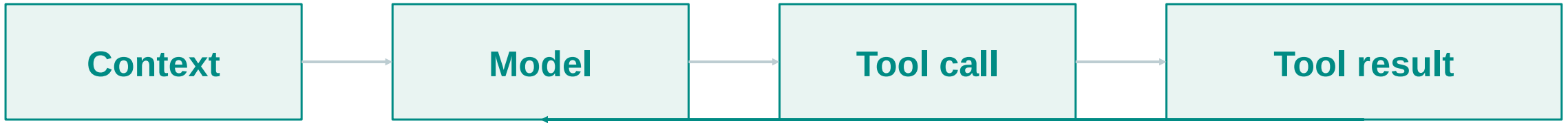
Test suite

Four runs, two destinations. Counts from `xcrun xcresulttool get test-results summary`, not console grep.

#	Destination	Mode	Result
1	iPhone 17	default (parallel)	157 passed / 0 failed / 0 skipped
2	iPhone 11 Pro Max	default (parallel)	157 / 0 / 0
3	iPhone 17	<code>-parallel-testing-enabled NO</code>	157 / 0 / 0
4	iPhone 17	<code>-only-testing the two new classes</code>	32 / 0 / 0

An agent is a loop

A model decides. The execution environment runs tools and returns results.



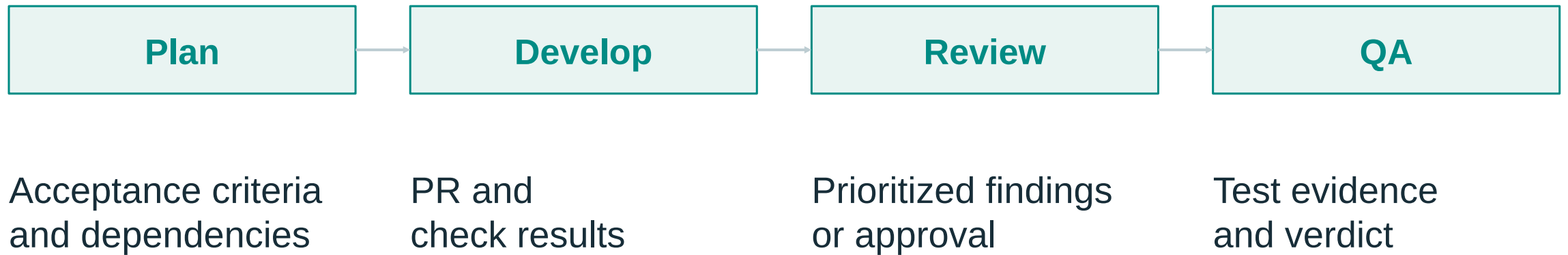
Result becomes new context for the next decision

The harness manages the loop, permissions and tool execution.

A skill supplies instructions. The host supplies execution.

One map, four responsibilities

Current documented workflow · Full steps in the backup



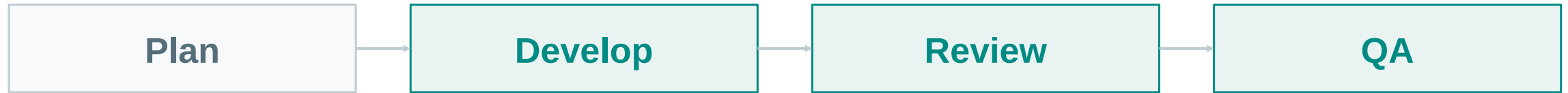
Orchestrator Clarify decisions, pass artifacts, choose the next stage

Project profile Shared commands and constraints

Preparation and completion checks surround this route. PIPELINE.md documents the instructions.

Changed code needs fresh evidence

Current documented workflow · Both return routes include code review



Review findings

Develop → Review

QA findings

Develop → Review → QA

Rule 11

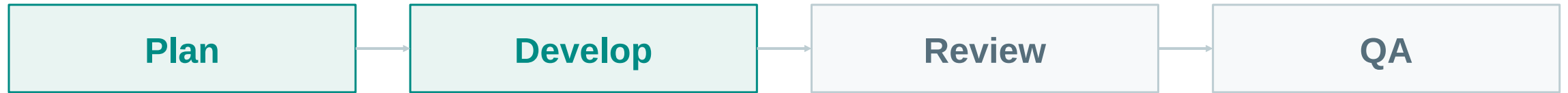
Review QA fixes
before retesting

Added 26 August 2026

Up to 3 iterations per repair loop, then escalate unresolved blockers. These are prompt instructions.

32 new tests. One delivery route.

PR #198 · ProgressStore and TestSession coverage



125 → 157

Tests for cumulative progress
and terminal session paths

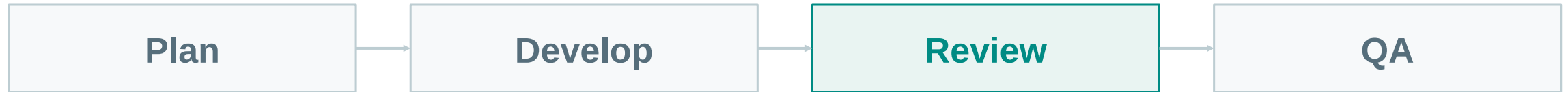
Scope

Tests + project profile
No application code
changes

The historical run used role agents. The current skill packaging appears later in this talk.

Review improved the patch

Two review passes before the first QA pass



Review 1

Conditional approval
with quality findings

✅ **APPROVAL** (conditional — items 1 and 2 are worth one short follow-up pass; nothing blocks merge)

Review 2

Developer follow-up
then APPROVAL

Code Review — Iteration 2

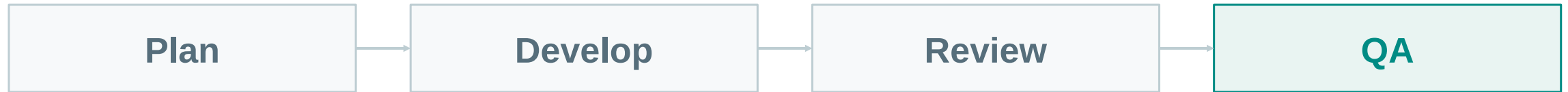
Re-review of the fix pass `ae659e9..e19273c` (+26 / -2).

✅ **APPROVAL**

The first review was not BLOCKED. QA made the later blocking decision.

Green tests. Weak evidence.

QA Pass 1: BLOCKED on three in-scope findings



What could still be wrong?

Positive totals can predate the test

Shared storage can leak state

Exit paths need separate checks

QA — Pass 1

Verdict: 🚫 BLOCKED on three in-scope findings.

Test suite

Four runs, two destinations. Counts from `xcrun xcrresulttool get test-results summary`, not console grep.

#	Destination	Mode	Result
1	iPhone 17	default (parallel)	157 passed / 0 failed / 0 skipped
2	iPhone 11 Pro Max	default (parallel)	157 / 0 / 0
3	iPhone 17	<code>-parallel-testing-enabled NO</code>	157 / 0 / 0
4	iPhone 17	<code>-only-testing the two new classes</code>	32 / 0 / 0

Passing the suite establishes the recorded outcomes; it does not establish the quality of every assertion.

Prove the change, isolate the state

The assertion and the fixture address connected concerns

Illustrative dirty store

Before 500 seconds

After 500 seconds

after > 0 PASS

after > before FAIL

The repair

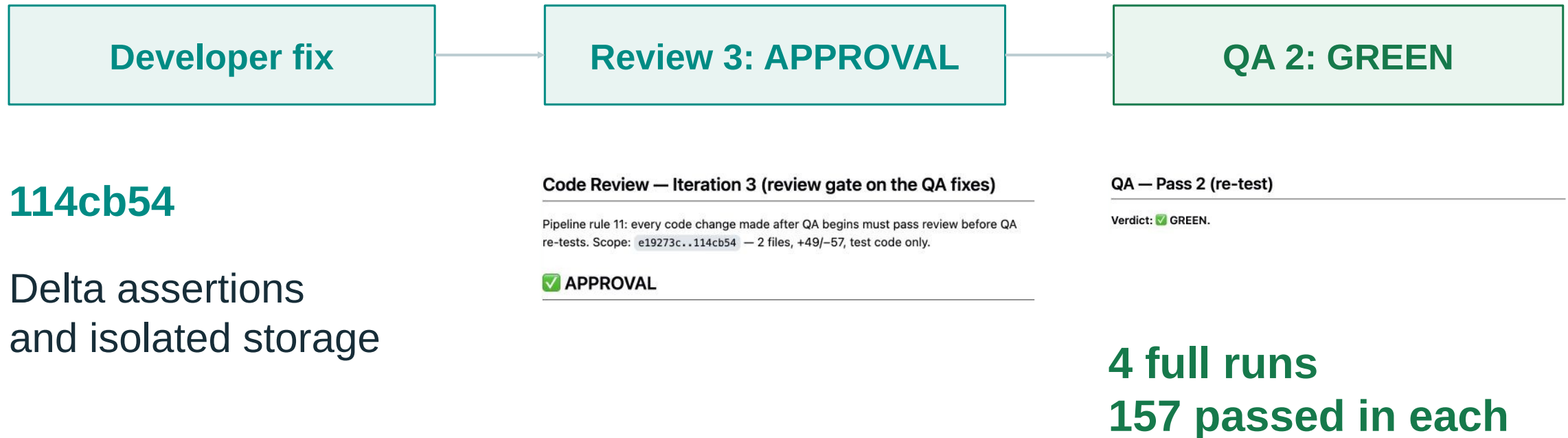
- 1 Compare before and after
- 2 Give each test its own store

With before = 0, both comparisons are numerically equivalent.
Isolation removes the original risk.

500 seconds is an illustration, not a measured result. Review 3 explicitly records the zero-baseline caveat.

Fix → review → QA

PR #198 · The repaired tests went through the review gate



PR open at capture, 6 September 2026. QA green permits completion checks; it does not establish merge.

The observed cost

The same ProgressStore coverage task

Measure	Recorded result
Subagent launches	9
Summed stage time	About 1 h 31 min
Subagent tokens	1,097,667

Planner ×1 Developer ×3
Reviewer ×3 QA ×2

Orchestrator tokens excluded. No matched single-agent baseline.

Measured: subagent tokens

Step	Role	Tokens	Tool calls	Wall clock
1	task-planner	113,654	24	7m 10s
3	developer	159,422	59	24m 52s
4 · iter 1	code-reviewer	134,464	30	7m 57s
4 · fix	developer	74,767	19	4m 00s
4 · iter 2	code-reviewer	95,164	20	5m 41s
5	qa-tester	164,345	37	12m 31s
6a	developer	96,169	30	8m 32s
6b	code-reviewer	118,791	36	6m 43s
6c	qa-tester	140,891	47	13m 29s
Total	9 launches	1,097,667	302	1h 31m

How the repository works today

Current structure · 13 September 2026

Canonical skills

skills/<role>/SKILL.md
Procedure + expected output

Thin host adapters

Separate stage agents
Optional model / reasoning settings

Main conversation

Orchestrator stays with the user

Shared context

Project profile + explicitly read role memory

The measured run predates this packaging. Its effect on cost and quality has not been measured.

Define what each stage hands over

Start with one workflow in your own repository

Stage	Concrete result for the next stage
Planner	Acceptance criteria, dependencies and issues
Developer	PR / diff, changed files and check results
Reviewer	Prioritized findings or explicit APPROVAL
QA	Test evidence, classified bugs, GREEN / BLOCKED

The orchestrator verifies the artifacts and routes the next stage. Dry-run uses local drafts.

Three things to take back

01 Define the output of each stage

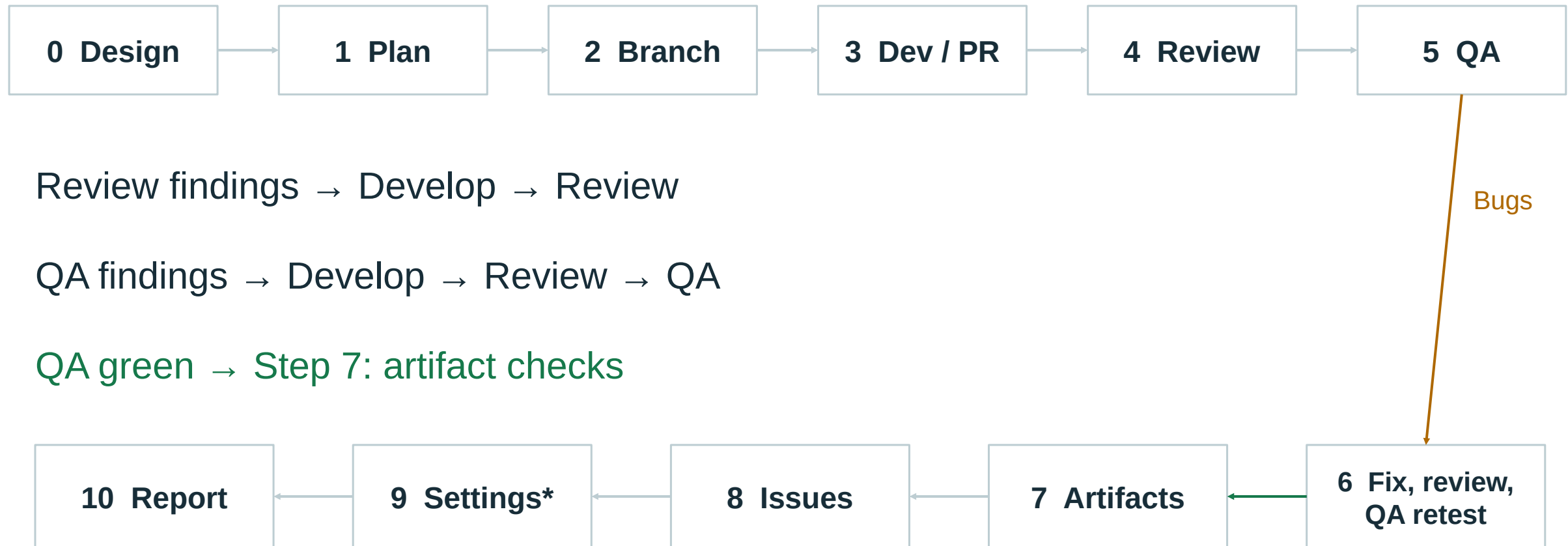
02 Pass artifacts that others can verify

03 Repeat verification after code changes

github.com/TeleginS/agent-orchestration

Full workflow and transition conditions

Backup · Current documented pipeline



Clarify design first. Both repair loops: max 3 iterations, then escalate. * Settings are optional.

The zero-baseline caveat

Backup · Actual diff and Review 3

```
@@ -53,46 +58,53 @@ final class TestSessionProgressTests: XCTestCase {
53 58 + func testCompleteTestAccumulatesTime() {
54 59     session.start(.quick(from: bank))
55 60     letSessionRun()
61 + let before = progressStore.totalTestSeconds
56 62
57 63     session.completeTest()
58 64
59 - XCTAssertGreaterThan(progressStore.totalTestSeconds, 0)
65 + XCTAssertGreaterThan(progressStore.totalTestSeconds, before)
60 66 }
```

1. The  [BUG] Five terminal-path tests assert an absolute totalTestSeconds instead of a delta #199 delta form is correct, but credit it accurately. With the hermetic suite from  [BUG] New Progress tests are not isolated: UserDefaults.standard instead of a dedicated suite #200, `before` is `0` in all five tests — always — so `> before` is numerically identical to the old `> 0`. The

In the repaired fixture

before = 0

after > 0

after > before

Numerically equivalent
for these five tests

The dirty-store example describes the original risk, not the repaired fixture.